PYTHON KURSU • HAFTA 6 • GÜN 1

JSON, XML ve API

Ders 11 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- JSON metnini Python sözlüğüne ve geri çevirebileceksiniz.
- XML belgesinden ElementTree ile veri çekebileceksiniz.
- API, uç nokta, parametre ve durum kodu kavramlarını açıklayabileceksiniz.
- requests ile internetten JSON ve XML verisi alabileceksiniz.
- TCMB döviz kurlarını ve bir hava durumu API'sini kullanan program yazabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 10'in tekrarı
0:10–1:25	75 dk	Blok 1: JSON • json modülü ve dosyalar • XML ve ElementTree • API nedir
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: requests • durum kodları ve hatalar • TCMB döviz kurları • Open-Meteo hava durumu • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 10'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ne yapar?

```
python -m pip install requests
```

SORU 2 • Ekranda ne yazar?

```
import random
print(random.randint(1, 1))
```

SORU 3 • Dosyada ne kalır?

```
with open("a.txt", "w") as f:
    f.write("x")
with open("a.txt", "w") as f:
    f.write("y")
```

SORU 4 • Neden hata verir?

```
# dosya adı: random.py
import random
random.randint(1, 6)
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 2: harcama günlüğü

```python
from datetime import date
aciklama, tutar = "Market", 245.90
with open("harcamalar.txt", "a", encoding="utf-8") as f:
    f.write(f"{date.today()};{aciklama};{tutar}\n")
toplam = 0
with open("harcamalar.txt", encoding="utf-8") as f:
    for satir in f:
        toplam += float(satir.strip().split(";")[2])
print(f"Toplam harcama: {toplam:.2f} TL")
```

```
ÇIKTI
Toplam harcama: 245.90 TL
```

Kendi biçimimizi (tarih;açıklama;tutar) uydurduk. Başkası farklı bir biçim seçseydi? **Ortak bir biçim gerekir: JSON.**

BLOK 1 • 0:10–1:25 • 75 DK

Veri biçimleri ve API

25 dk **JSON**	10 dk **JSON dosyaları**	20 dk **XML**	20 dk **API nedir?**

BLOK 1 • JSON • 25 DK

JSON: programlar arası ortak veri dili

kisi.json

```
{
  "ad": "Ayşe Yılmaz",
  "yas": 34,
  "ogrenci": false,
  "notlar": [85, 72, 90],
  "adres": {"sehir": "Bursa", "posta_kodu": "16000"},
  "telefon": null
}
```

JSON	Python
{ } nesne	dict
[ ] dizi	list
"metin"	str
34, 1.5	int, float
true / false	True / False
null	None

BLOK 1 • JSON • 25 DK

json.loads: metin → sözlük • json.dumps: sözlük → metin

```
import json

metin = '{"ad": "Ayşe", "yas": 34, "ogrenci": false, "notlar": [85, 72]}'
kisi = json.loads(metin)
print(type(kisi), kisi["ad"], kisi["ogrenci"])
kisi["notlar"].append(90)
geri = json.dumps(kisi, ensure_ascii=False)
print(geri)
```

```
ÇIKTI
<class 'dict'> Ayşe False
{"ad": "Ayşe", "yas": 34, "ogrenci": false, "notlar": [85, 72, 90]}
```

BLOK 1 • JSON • 25 DK

indent: JSON’u okunaklı yazdırmak

```python
import json

kisi = {"ad": "Ayşe", "yas": 34, "ogrenci": False,
        "notlar": [85, 72, 90], "telefon": None}
print(json.dumps(kisi, ensure_ascii=False, indent=2))
```

```
ÇIKTI
{
  "ad": "Ayşe",
  "yas": 34,
  "ogrenci": false,
  "notlar": [
    85,
    72,
    90
  ],
  "telefon": null
}
```

BLOK 1 • JSON DOSYALARI • 10 DK

JSON dosyası: yapıyı koruyarak kaydetmek

```
import json

ayarlar = {"tema": "koyu", "dil": "tr", "yazi_boyutu": 14, "son_dosyalar": ["rapor.txt"]}
with open("ayarlar.json", "w", encoding="utf-8") as f:
    json.dump(ayarlar, f, ensure_ascii=False, indent=2)

with open("ayarlar.json", encoding="utf-8") as f:
    okunan = json.load(f)
print(okunan["tema"], okunan["yazi_boyutu"] + 2)
```

```
ÇIKTI
koyu 16
```

dump / load dosyayla, **dumps / loads** (s = string) metinle çalışır.

BLOK 1 • JSON • 25 DK

İç içe JSON: sözlükler listesi en yaygın yapı

```python
import json

veri = json.loads('''{"sinif": "10-A", "ogrenciler": [
  {"ad": "Ali", "notlar": [70, 85]},
  {"ad": "Ece", "notlar": [92, 88]}]}''')
print(veri["sinif"], len(veri["ogrenciler"]), "öğrenci")
for ogr in veri["ogrenciler"]:
    ort = sum(ogr["notlar"]) / len(ogr["notlar"])
    print(f"{ogr['ad']}: {ort:.1f}")
```

```
ÇIKTI
10-A 2 öğrenci
Ali: 77.5
Ece: 90.0
```

Okuma tekniği

Her adımda “elimde ne var?” diye sorun: veri bir sözlük → ["ogrenciler"] bir liste → döngüyle her öğe bir sözlük → ["notlar"] bir liste.

BLOK 1 • JSON • SIK YAPILAN HATALAR

JSON hataları ve anlamları

Hatalı kod	Hata mesajı	Çözüm
`json.loads("{'ad': 'Ali'}")`	`json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)`	JSON’da çift tırnak zorunlu
`json.loads('{"a": 1,}')`	`json.decoder.JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 9 (char 8)`	Sondaki virgülü silin
`json.dumps({1, 2})`	`TypeError: Object of type set is not JSON serializable`	Küme JSON’da yok: list(küme)
`d["soyad"]`	`KeyError: 'soyad'`	d.get("soyad")

BLOK 1 • XML • 20 DK

XML: etiketlerle yapılandırılmış veri

TCMB kurları (kısaltılmış, 20.03.2018)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Tarih_Date Tarih="20.03.2018" Bulten_No="2018/56">
  <Currency Kod="USD" CurrencyCode="USD">
    <Unit>1</Unit>
    <Isim>ABD DOLARI</Isim>
    <ForexBuying>3.9356</ForexBuying>
    <ForexSelling>3.9427</ForexSelling>
  </Currency>
  <Currency Kod="EUR" CurrencyCode="EUR">
    …
  </Currency>
</Tarih_Date>
```

Parça	Örnek
etiket (eleman)	`<Isim>…</Isim>`
öznitelik	`Kod="USD"`
metin	`3.9356`
kök	`<Tarih_Date>`

> **Benzetme: iç içe kutular**
>
> Büyük kutu (kök) içinde döviz kutuları, her döviz kutusunun içinde etiketli küçük kutular.

BLOK 1 • XML • 20 DK

ElementTree: XML'den veri çekmek

```python
import xml.etree.ElementTree as ET

kok = ET.parse("kurlar_ornek.xml").getroot()
print("Bülten tarihi:", kok.get("Tarih"))
for doviz in kok.findall("Currency"):
    kod = doviz.get("Kod")
    isim = doviz.find("Isim").text
    satis = float(doviz.find("ForexSelling").text)
    print(f"{kod} {isim:<11} {satis:.4f} TL")
```

```
ÇIKTI
Bülten tarihi: 20.03.2018
USD ABD DOLARI  3.9427 TL
EUR EURO        4.8604 TL
```

Yöntem	Ne verir?
`kok.get("Tarih")`	öznitelik değeri
`kok.findall("Currency")`	eşleşen elemanların listesi
`doviz.find("Isim").text`	ilk eşleşenin metni

BLOK 1 • XML • 20 DK

JSON mu, XML mi?

	JSON	XML
Görünüm	{"kod": "USD", "satis": 3.94}	<Currency Kod="USD"><Satis>3.94</Satis>…
Boyut	kısa	etiketler yüzünden uzun
Tipler	sayı, bool, null var	her şey metin
Python'da	json.loads → dict/list	ElementTree → eleman ağacı
Nerede?	web API'lerinin çoğu, ayar dosyaları	bankacılık, e-fatura, RSS, eski sistemler

Bugünkü uygulamalarda ikisi de var: **TCMB → XML**, **Open-Meteo → JSON**.

BLOK 1 • API NEDIR? • 20 DK

API: programların birbirine soru sorduğu kapı

> **Benzetme: restoranda garson**
>
> Mutfağa (sunucu) siz girmezsiniz. Garsona (API) menüdeki bir siparişi (istek) verirsiniz, garson tabağı (yanıt) getirir. Menü dışı sipariş “bizde yok” (404) yanıtı alır.

Kavram	Açıklama
İstemci	isteği gönderen (bizim programımız)
Sunucu	veriyi tutan bilgisayar
Uç nokta (endpoint)	belirli bir verinin adresi
Parametre	isteği özelleştiren ?ad=değer
Yanıt	JSON/XML veri + durum kodu

```
https://api.open-meteo.com/v1/forecast?latitude=41.01&longitude=28.98&current=temperature_2m
└──────── sunucu ──────┘└─ uç nokta ─┘└──────────────────── parametreler ──────────────────────┘
```

BLOK 1 • API NEDIR? • 20 DK

HTTP durum kodları: yanıtın “nasıl gittiği”

Kod	Anlamı	Ne yapmalı?
`200 OK`	başarılı, veri geldi	veriyi işle
`301 / 302`	adres taşındı	requests genellikle kendisi takip eder
`400 Bad Request`	istek hatalı (eksik/yanlış parametre)	parametreleri kontrol et
`401 / 403`	yetki yok (anahtar eksik/yanlış)	API anahtarını kontrol et
`404 Not Found`	böyle bir adres yok	URL’yi kontrol et
`429 Too Many Requests`	çok sık istek	bekle, daha seyrek iste
`500 / 503`	sunucuda sorun	sonra tekrar dene

Kural: **2xx başarı • 4xx sizin hatanız • 5xx sunucunun hatası**

1:25–1:40 • 15 DK

Mola

Moladan sonra: Python’dan internete nasıl istek gönderilir? (requests kurulu değilse şimdi kurun: python -m pip install requests)

BLOK 2 • 1:40–2:55 • 75 DK

requests ile canlı veri

20 dk
requests

15 dk
TCMB kurları

15 dk
Hava durumu

20 dk
Alıştırmalar

BLOK 2 • REQUESTS • 20 DK

requests.get: istek gönder, yanıtı al

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
parametreler = {"latitude": 41.01, "longitude": 28.98, "current": "temperature_2m"}
yanit = requests.get(url, params=parametreler, timeout=10)
print("Durum kodu:", yanit.status_code)
veri = yanit.json()
print("Sıcaklık:", veri["current"]["temperature_2m"], "°C")
print(yanit.url)
```

```
ÖRNEK ÇIKTI (değerler o güne göre değişir)
Durum kodu: 200
Sıcaklık: 17.4 °C
https://api.open-meteo.com/v1/forecast?latitude=41.01&longitude=28.98&current=temperature_2m
```

BLOK 2 • REQUESTS • 20 DK

İnternet her zaman çalışmaz: hataları yakalayın

```python
import requests

def veri_getir(url, **parametreler):
    try:
        yanit = requests.get(url, params=parametreler, timeout=10)
        yanit.raise_for_status()
        return yanit.json()
    except requests.exceptions.HTTPError:
        print("Sunucu hata kodu döndürdü:", yanit.status_code)
    except requests.exceptions.ConnectionError:
        print("Bağlantı kurulamadı.")
    except requests.exceptions.Timeout:
        print("Sunucu zamanında yanıt vermedi.")
    return None

print(veri_getir("https://api.open-meteo.com/v1/olmayan_uc_nokta"))
print(veri_getir("https://bu-adres-yok-12345.com"))
```

```
ÇIKTI
Sunucu hata kodu döndürdü: 404
None
Bağlantı kurulamadı.
None
```

BLOK 2 • REQUESTS • 20 DK

API kullanırken iyi davranış kuralları

1 • TIMEOUT

Her isteğe süre sınırı

requests.get(…, timeout=10) sonsuz beklemeyi önler.

2 • SEYREK İSTE

Döngüde istek yağdırmayın

Aynı veriyi bir kez alıp saklayın; 429 ve engellenme riski azalır.

3 • ANAHTARI SAKLA

API anahtarı koda yazılmaz

Anahtar şifre gibidir; GitHub'a giden koda yazılmaz. Ortam değişkeni kullanın.

4 • KOŞULLARI OKU

Kullanım koşulları ve lisans

Kullanım ve atıf kurallarını sağlayıcının sayfasında okuyun.

BLOK 2 • REQUESTS • 20 DK

Her API programının dört adımı

ADIM 1

İstek

`requests.get(url, params)`

ADIM 2

Kontrol

`raise_for_status()`

ADIM 3

Ayrıştır

`.json()` ya da `ET.fromstring`

ADIM 4

Kullan

`sözlük / liste işlemleri`

Neden bu sıra?

Önce paketi alırsınız, sonra hasarlı mı diye bakarsınız, sonra açarsınız, en son içindekini kullanırsınız. Hasarlı paketi açmaya çalışmak (404 yanıtını JSON sanmak) hataya yol açar.

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 1 • 12 DK

TCMB günlük döviz kurları (gerçek veri, XML)

tcmb_kur.py (kısaltılmış)

```python
def tcmb_kurlari():
    yanit = requests.get(TCMB_URL, timeout=10)
    yanit.raise_for_status()
    kok = ET.fromstring(yanit.content)
    kurlar = {}
    for doviz in kok.findall("Currency"):
        satis = doviz.find("ForexSelling").text
        if satis:
            birim = int(doviz.find("Unit").text)
            isim = doviz.find("Isim").text
            kurlar[doviz.get("Kod")] = (isim, float(satis) / birim)
    return kok.get("Tarih"), kurlar

tarih, kurlar = tcmb_kurlari()
```

ÖRNEK ÇIKTI

```
Bülten: 20.03.2018 (2 döviz)
USD (ABD DOLARI): 3.9427 TL
EUR (EURO): 4.8604 TL
250 USD = 985.67 TL
```

20.03.2018 bülteninden; derste o günün kurları gelir.

BLOK 2 • UYGULAMA 1 • SATIR SATIR

Veri nereden nereye aktı?

Adım	Kod	Sonuç
1	`requests.get(…today.xml)`	`XML metni (baytlar)`
2	`ET.fromstring(yanit.content)`	`kök eleman: Tarih_Date`
3	`kok.findall("Currency")`	`her döviz için bir eleman`
4	`find("ForexSelling").text`	`"3.9427" (metin)`
5	`float(satis) / birim`	`3.9427 (sayı)`
6	`kurlar[kod] = (isim, kur)`	`{"USD": ("ABD DOLARI", 3.9427), …}`

Neden fonksiyon?

tcmb_kurlari() bir kez yazıldı; döviz çevirici, Streamlit uygulaması (Ders 12) ya da günlük rapor, hepsi aynı fonksiyonu çağırabilir.

BLOK 2 • UYGULAMA 1 • EK

Geçmiş günlerin kurları: adresi tarihten üretmek

```python
from datetime import date

def bulten_adresi(tarih):
    return f"https://www.tcmb.gov.tr/kurlar/{tarih:%Y%m}/{tarih:%d%m%Y}.xml"

print(bulten_adresi(date(2018, 3, 20)))
print(bulten_adresi(date(2026, 10, 23)))
```

```
ÇIKTI
https://www.tcmb.gov.tr/kurlar/201803/20032018.xml
https://www.tcmb.gov.tr/kurlar/202610/23102026.xml
```

Bu adres **tcmb_kurlari()** fonksiyonuna parametre olarak verilirse herhangi bir iş gününün kurları alınabilir.

BLOK 2 • GERÇEK HAYATTAN UYGULAMA 2 • 12 DK

Hava durumu: Open-Meteo (gerçek veri, JSON)

hava.py (kısaltılmış)

```
def hava_durumu(enlem, boylam, gun=3):
    parametreler = {"latitude": enlem, "longitude": boylam,
                    "current": "temperature_2m,wind_speed_10m",
                    "daily": "temperature_2m_max,temperature_2m_min",
                    "timezone": "Europe/Istanbul", "forecast_days": gun}
    yanit = requests.get(URL, params=parametreler, timeout=10)
    yanit.raise_for_status()
    return yanit.json()

veri = hava_durumu(41.01, 28.98)
gunluk = veri["daily"]
for t, yuk, dus in zip(gunluk["time"], gunluk["temperature_2m_max"],
                       gunluk["temperature_2m_min"]):
    print(f"{t}: {dus:>5.1f} / {yuk:>5.1f} °C")
```

ÖRNEK ÇIKTI

```
İstanbul şu an: 17.4 °C
Rüzgâr: 12.6 km/sa
2026-10-26:  13.2 /  19.8 °C
2026-10-27:  12.7 /  18.1 °C
2026-10-28:  11.9 /  16.5 °C
```

Değerler örnek; derste o anki veri gelir.

BLOK 2 • UYGULAMA 2 • SATIR SATIR

JSON yanıtının yapısı

```
yanıtın iskeleti (örnek değerler)
{
  "latitude": 41.0, "longitude": 28.98,
  "current": {"time": "…", "temperature_2m": 17.4,
              "wind_speed_10m": 12.6},
  "daily": {
    "time": ["2026-10-26", "2026-10-27", "…"],
    "temperature_2m_max": [19.8, 18.1, "…"],
    "temperature_2m_min": [13.2, 12.7, "…"]
  }
}
```

Paralel listeler

time[0], max[0] ve min[0] aynı güne aittir. zip bu üç listeyi fermuar gibi birleştirip her turda bir günün üç değerini verir.

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 20 DK

Alıştırma 1, 2 ve 3

KOLAY • 5 dk

1 • Kitaplık JSON’u

3 kitaplık bir listeyi (ad, yazar, yıl) kitaplar.json dosyasına kaydedin; dosyadan okuyup 2000’den sonra yayımlananları yazdırın.

ORTA • 6 dk

2 • Döviz çevirici

tcmb_kurlari() fonksiyonunu kullanarak kullanıcıdan döviz kodu (USD, EUR, GBP…) ve TL tutarı alın; kaç birim döviz alınabileceğini yazdırın. Olmayan kodda uyarı.

ZOR • 7 dk

3 • En sıcak şehir

hava_durumu() ile SEHIRLER sözlüğündeki üç şehrin anlık sıcaklığını alın; en sıcak şehri ve sıcaklığını yazdırın. Ağ hatasında program çökmesin.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: kitaplık JSON'u

```
import json

kitaplar = [{"ad": "Tutunamayanlar", "yazar": "Oğuz Atay", "yil": 1972},
            {"ad": "Puslu Kıtalar Atlası", "yazar": "İhsan Oktay Anar", "yil": 1995},
            {"ad": "Kafamda Bir Tuhaflık", "yazar": "Orhan Pamuk", "yil": 2014}]
with open("kitaplar.json", "w", encoding="utf-8") as f:
    json.dump(kitaplar, f, ensure_ascii=False, indent=2)
with open("kitaplar.json", encoding="utf-8") as f:
    for k in json.load(f):
        if k["yil"] > 2000:
            print(f"{k['ad']} ({k['yazar']}, {k['yil']})")
```

```
ÇIKTI
Kafamda Bir Tuhaflık (Orhan Pamuk, 2014)
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 2: döviz çevirici

```
tarih, kurlar = tcmb_kurlari()
kod = input("Döviz kodu (USD, EUR…): ").strip().upper()
if kod not in kurlar:
    print("Bu döviz bültende yok.")
else:
    tl = float(input("TL tutarı: ").replace(",", "."))
    isim, kur = kurlar[kod]
    print(f"{tl:,.2f} TL ≈ {tl / kur:,.2f} {kod}")
```

```
ÖRNEK: eur, 1000
Döviz kodu (USD, EUR…): eur
TL tutarı: 1000
1,000.00 TL ≈ 205.74 EUR
Kur: 4.8604 (EURO)
```

Kur 20.03.2018 bülteninden.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 3: en sıcak şehir

```
sicakliklar = {}
for sehir, (enlem, boylam) in SEHIRLER.items():
    try:
        veri = hava_durumu(enlem, boylam, gun=1)
        sicakliklar[sehir] = veri["current"]["temperature_2m"]
    except requests.exceptions.RequestException:
        print(f"{sehir} için veri alınamadı.")
en_sicak = max(sicakliklar, key=sicakliklar.get)
```

```
ÖRNEK ÇIKTI
İstanbul: 17.4 °C
Ankara: 17.4 °C
İzmir: 17.4 °C
En sıcak: İstanbul (17.4 °C)
```

Ağ hatasında o şehir atlanır; program çökmez.

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- JSON ≈ Python sözlüğü: **loads/dumps** metin, **load/dump** dosya.
- XML: **ET.fromstring**, **findall**, **find(…).text**, **get("öznitelik")**.
- API: URL + parametreler → yanıt (JSON/XML) + durum kodu (200, 404, 500…).
- **requests.get(url, params=…, timeout=10)**, **raise_for_status()**, **.json()**.
- Ağ kodunu her zaman **try/except** içine alın.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kur takip günlüğü

tcmb_kurlari() ile USD ve EUR kurlarını alıp tarihle birlikte kurlar.json dosyasındaki listeye ekleyin (oku → ekle → yaz). Birkaç gün çalıştırıp değişimi yazdırın.

2 • ZORUNLU

Şehrim için hava raporu

Kendi şehrinizin koordinatlarıyla 7 günlük tahmini alın; en sıcak ve en soğuk günü, haftalık ortalamayı yazdırın.

3 • BONUS

GitHub hesabı

Ders 13'te uygulamamızı internette yayınlayacağız. github.com'da ücretsiz bir hesap açın ve kullanıcı adınızı getirin.

Teslim: ders11_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 6 • GÜN 2

Streamlit 1

Programlarımızın hepsi terminalde çalıştı. Gelecek ders tarayıcıda çalışan bir arayüz yapacağız:

- Streamlit kurulumu
- Başlık, metin, girdi kutuları, düğmeler
- Tablolar ve basit grafikler
- Döviz kuru gösterici web uygulaması

Önizleme

```
import streamlit as st

st.title("Merhaba Streamlit")
ad = st.text_input("Adınız")
st.write("Hoş geldin,", ad)
```

```
(tarayıcıda açılan
 sayfada görünür)
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
json modülü	docs.python.org/3/library/json.html
xml.etree.ElementTree	docs.python.org/3/library/xml.etree.elementtree.html
requests belgeleri	requests.readthedocs.io
TCMB gösterge niteliğindeki kurlar	tcmb.gov.tr/kurlar/today.xml
Open-Meteo API belgeleri	open-meteo.com/en/docs